LongEval 2025 Web-Retrieval: A Two-Stage BM25 and Neural Reranking Pipeline under Temporal Drift

Lawrence Obiuwevwi
Department of Computer Science
Old Dominion University
lobiu001@odu.edu

Abstract

This report presents a two-stage information retrieval pipeline for the LongEval 2025 Web Retrieval task, focusing on robustness under temporal drift between monthly web snapshots. We implement a BM25 first-stage retriever and rerank the top candidates using five supervised/neural ranking approaches: XGBoost, Naive Bayes, RankNet, ListNet, and a transformer CrossEncoder. Experiments are performed across six train→test temporal mappings (2022→2023). Results show consistent gains over BM25 from tree-based reranking (XGBoost) across all months, while the transformer CrossEncoder exhibits strong early-month performance but unstable behavior under later-month domain drift and/or indexing mismatches. We also document key engineering challenges encountered when indexing mixed-format collections (TREC vs. JSON) and handling docno consistency for cross-temporal inference.

1 Introduction

Modern information retrieval systems are increasingly deployed in dynamic, real-world environments where the underlying document collections, user queries, and relevance signals evolve over time. This phenomenon, commonly referred to as *temporal drift*, poses a fundamental challenge to retrieval robustness: models trained on historical data may degrade significantly when applied to future collections whose lexical distributions, topical focus, and user intent have shifted (1).
The **LongEval 2025** shared task was explicitly designed to study this challenge by evaluating retrieval systems under **cross-temporal generalization**. In LongEval, models are trained on web snapshots from earlier months and evaluated on later snapshots, reflecting realistic deployment scenarios faced by commercial search engines and archival retrieval systems (2). The French track uses a large-scale web corpus derived from the Qwant search engine, with monthly releases that capture natural changes in web content and user behavior over time.

Objective. The primary objective of this project is to assess whether supervised and neural reranking models can maintain performance under temporal drift when integrated into a standard multi-stage retrieval pipeline. Specifically, we aim to determine:

- whether neural rerankers consistently improve upon a lexical BM25 baseline across multiple future months,
- which reranking approaches are more stable under distribution shift, and
- how temporal mismatch between training and test data impacts ranking effectiveness.

Key Challenges. Addressing these objectives introduces several non-trivial challenges. First, relevance judgments are inherently time-dependent: click-based signals collected during training may not align with future user intent (3). Second, neural ranking models are known to be sensitive to domain and distribution shifts, often overfitting to training-period statistics (4). Finally, large-scale web retrieval requires efficient architectures that balance effectiveness with computational constraints, motivating the use of multi-stage pipelines rather than end-to-end neural retrieval (5).

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

Approach. To address these challenges, we adopt a **two-stage retrieval architecture**. An initial BM25 retriever generates a candidate set of documents, which are then reranked using a range of supervised and neural ranking models, including tree-based, pairwise, listwise, and transformer-based approaches. By evaluating this pipeline across six cross-temporal train–test splits, we provide a systematic analysis of reranking robustness under temporal drift.

2 Related Work

Research in information retrieval has progressively shifted from purely lexical matching toward learned ranking models that leverage representation learning and neural architectures. Early neural IR work formalized the transition from term-matching heuristics to learned relevance functions, providing a unifying view of neural ranking paradigms, training objectives, and evaluation challenges (1). This shift has been particularly influential in web-scale retrieval, where semantic generalization is required to handle vocabulary mismatch and evolving user intent.

Despite the effectiveness of neural models, fully end-to-end neural retrieval remains computationally expensive for large document collections. As a result, **multi-stage retrieval pipelines** have become the dominant architecture in both academic benchmarks and industrial systems. In these pipelines, a fast lexical retriever such as BM25 generates an initial candidate set, which is subsequently refined using more expensive neural rerankers (5). This design balances efficiency and effectiveness and underpins many modern evaluation tracks, including TREC Deep Learning and LongEval.

Transformer-based reranking models, particularly CrossEncoders, have demonstrated strong effectiveness by jointly encoding query–document pairs and learning fine-grained relevance interactions (6; 4). However, their sensitivity to domain and distribution shifts has been observed in several studies, motivating investigations into robustness and generalization. To mitigate computational and generalization challenges, hybrid approaches combine lexical signals with contextualized representations, such as COIL, which integrates exact term matching with contextual embeddings (7).

Beyond transformer models, supervised learning-to-rank methods remain competitive and attractive due to their stability and interpretability. Tree-based and listwise approaches have been shown to provide strong gains when trained on carefully engineered features and integrated into multi-stage pipelines (8). These methods are particularly relevant in longitudinal or cross-temporal settings, where robustness across evolving collections is critical.

Collectively, this body of work motivates the present study: evaluating how different reranking paradigms, lexical, feature-based, and neural, behave under temporal drift when applied to future web snapshots. By grounding our approach in established multi-stage retrieval designs, we focus on practical robustness rather than isolated peak performance.

3 Dataset

3.1 LongEval Collection Overview

This study uses the official French-language collection released for the **CLEF LongEval 2025 Information Retrieval Challenge**, which is designed to evaluate retrieval robustness under temporal distribution shift (2). The dataset is derived from the Qwant search engine and consists of longitudinal web snapshots that capture natural changes in document content, availability, and user search behavior over time.

The collection is partitioned into temporally ordered training and test sets. The *LongEval Train Collection* contains historical snapshots used for model training and validation, while the *LongEval Test Collection* consists of future snapshots reserved for evaluation. This strict temporal separation enforces a cross-temporal setting in which models trained on earlier data are applied to later, unseen distributions, closely mirroring real-world deployment scenarios.

3.2 Documents and Relevance Judgments

Training documents are provided in two complementary representations. A TREC-formatted version supports traditional inverted-index retrieval and integration with classical IR toolkits, while a JSON-formatted version exposes structured document fields and metadata suitable for feature-based and neural reranking. Relevance judgments (*qrels*) are supplied for each training month and associate query identifiers with relevant document identifiers based on click-derived relevance signals from Qwant. No additional subsampling is applied beyond spam filtering, allowing flexibility in preprocessing and feature design.

Test documents are distributed exclusively in JSON format and contain structured fields such as document identifiers, titles, body text, and auxiliary metadata. These documents do not include relevance judgments and are used solely for evaluation under temporal shift.

3.3 Queries and Temporal Drift

Queries are distributed independently of the document collections and are shared across months. Each query is identified by a stable query identifier and mapped to a short natural-language query string derived from trending user-issued searches. Although query identifiers and file formats remain fixed across time, query semantics naturally evolve as topics and user intent shift, contributing directly to the temporal drift studied in this task.

3.4 Temporal Mapping (Train → Test)

We evaluate six cross-temporal train–test mappings: 2022-06 → 2023-03, 2022-07 → 2023-04, 2022-08 → 2023-05, 2022-09 → 2023-06, 2022-10 → 2023-07, and 2022-11 → 2023-08. These pairings enable a systematic assessment of retrieval robustness under increasing temporal drift.

3.5 Metadata and Document Traceability

To support temporal analysis and document traceability, the dataset includes auxiliary SQLite databases that map document identifiers to URLs, timestamps, and the months in which documents appear. This metadata enables tracking document persistence and turnover across snapshots and is particularly useful for diagnosing retrieval failures caused by document disappearance, modification, or identifier inconsistencies.

3.6 Data Formats and Indexing Implications

A key practical challenge in this study arises from the use of heterogeneous document formats across training and test phases, which necessitates distinct indexing strategies and careful handling of document identifiers. Training documents follow the standard **TREC format** and are ingested using PyTerrier's `TRECCollectionIndexer`, while test documents are provided as structured **JSON objects** and require dictionary-based ingestion (e.g., `IterDictIndexer`) with explicit field selection and identifier mapping. As a result, training and test collections are indexed independently using separate pipelines.

Maintaining consistent mappings between query identifiers (`qid`) and document identifiers (`docno`) across retrieval and reranking stages is therefore critical. Any mismatch can silently invalidate relevance signals, a risk amplified in cross-temporal evaluation where collections share no common document space. While feature-based rerankers are relatively robust to minor structural inconsistencies, transformer-based CrossEncoders are highly sensitive to errors in text selection and identifier alignment, which can lead to severe performance degradation if not carefully validated.

3.7 Dataset Scale

As used in this project, the dataset comprises approximately **36 GB** of training data and **59 GB** of test data, with around **9,000 training queries** and **18 million documents**. The scale and structure of the collection impose realistic constraints on indexing, feature extraction, and reranking, making it well suited for evaluating multi-stage retrieval pipelines under temporal drift.

4 System Architecture

4.1 Two-Stage Retrieval Pipeline

Our retrieval pipeline follows a standard multi-stage design:

Stage 1: BM25 Retrieval Given a query q, retrieve top-K documents (here $K = 100$) using BM25.

Stage 2: Reranking Compute a feature set for each candidate (q, d) pair and apply reranking models to produce a new ordering.

4.2 BM25 Baseline

BM25 scoring is:

$$\text{BM25}(q,d) = \sum_{t \in q} \text{IDF}(t)\, \frac{f(t,d)(k_1+1)}{f(t,d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}. \tag{1}$$

4.3 Reranking Models

We evaluated the following rerankers.

XGBoost (Gradient-Boosted Trees)

$$\hat{y}(d) = \sum_{k=1}^{K} f_k(d). \tag{2}$$

Naive Bayes

$$P(\text{rel} \mid d) \propto P(\text{rel}) \prod_i P(w_i \mid \text{rel}). \tag{3}$$

RankNet (Pairwise)

$$P(d_i \succ d_j) = \sigma(s_i - s_j), \qquad \mathcal{L} = -\log P(d_i \succ d_j). \tag{4}$$

ListNet (Listwise)

$$P_{\text{model}}(d_i) = \frac{e^{s_i}}{\sum_j e^{s_j}}, \qquad \mathcal{L} = -\sum_i P_{\text{true}}(d_i) \log P_{\text{model}}(d_i). \tag{5}$$

CrossEncoder Transformer We use a joint query-document encoder:

$$s(q, d) = \text{MLP}(\text{CLS}(q, d)). \tag{6}$$

In our implementation, we used a multilingual MiniLM-family CrossEncoder and GPU batching.

4.4 Evaluation Metrics

We report standard IR metrics:

Precision@10 (P@10)

$$P@10 = \frac{\#\text{relevant in top 10}}{10}. \tag{7}$$

nDCG@10

$$\text{nDCG@10} = \frac{\text{DCG@10}}{\text{IDCG@10}}, \quad \text{DCG@10} = \sum_{i=1}^{10} \frac{rel_i}{\log_2(i+1)}. \tag{8}$$

Average Precision (AP)

$$AP = \frac{1}{R} \sum_{k=1}^{N} P@k \cdot rel_k. \tag{9}$$

Reciprocal Rank (RR)

$$RR = \frac{1}{\text{rank of first relevant}}. \tag{10}$$

4.5 Experimental Setup

All experiments were executed in a Linux-based environment with reproducible indexing and evaluation scripts.

4.6 Compute Resources

All experiments were conducted on a single workstation equipped with an NVIDIA RTX 4000 SFF Ada Generation GPU with 20 GB of VRAM. The system ran NVIDIA driver version 575.64.03 with CUDA 12.9 enabled. GPU acceleration was used primarily for transformer-based CrossEncoder inference, with batched processing employed to control memory usage and ensure stable execution during large-scale reranking.

4.7 Software Stack

The retrieval and reranking pipeline was implemented in Python 3.10 using PyTerrier as the primary framework for indexing, retrieval, feature extraction, and pipeline orchestration, with Apache Lucene providing the underlying inverted index and BM25 retrieval functionality. Supervised learning-to-rank models were implemented using XGBoost and scikit-learn, while neural reranking models, including RankNet, ListNet, and transformer-based CrossEncoders, were implemented in PyTorch. Multilingual MiniLM-family CrossEncoder models were loaded via the Sentence-Transformers library.

5 Results

Table 1 summarizes retrieval effectiveness across six cross-temporal train–test splits, comparing the BM25 baseline against five supervised and neural reranking models. Results are reported using standard IR metrics (P@10, nDCG@10, AP, and RR) and reflect performance on future-month test collections relative to models trained on earlier snapshots. Bold values indicate the best-performing model for each month and metric.

Beyond absolute effectiveness, the results provide insight into the relative robustness of different reranking paradigms under temporal drift. By evaluating performance trends across successive months, the table illustrates how learned ranking models vary in their ability to generalize to evolving document distributions, offering a basis for analyzing stability, degradation, and consistency in cross-temporal retrieval settings.

Table 1: Complete results across six cross-temporal splits.

Month	Model	P@10	nDCG@10	AP	RR
Month 1 (2022-06 → 2023-03)					
	BM25	0.04519	0.15880	0.12889	0.18474
	XGBoost	0.04646	0.16681	0.13576	0.19539
	NaiveBayes	0.04519	0.15869	0.12872	0.18440
	RankNet	0.00405	0.01574	0.01784	0.02802
	ListNet	0.04515	0.15872	0.12888	0.18479
	CrossEncoder	**0.05201**	**0.20473**	**0.17220**	**0.24405**
Month 2 (2022-07 → 2023-04)					
	BM25	0.05491	0.18890	0.15898	0.19861
	XGBoost	**0.05645**	**0.19730**	**0.16596**	**0.20861**
	NaiveBayes	0.05493	0.18888	0.15891	0.19852
	RankNet	0.00262	0.00676	0.01329	0.01521
	ListNet	0.05493	0.18892	0.15895	0.19856
	CrossEncoder	0.05491	0.18890	0.15898	0.19861
Month 3 (2022-08 → 2023-05)					
	BM25	0.05603	0.18734	0.15735	0.19841
	XGBoost	**0.05685**	**0.19433**	**0.16422**	**0.20829**
	NaiveBayes	0.05603	0.18733	0.15736	0.19838
	RankNet	0.05589	0.18697	0.15727	0.19806
	ListNet	0.05601	0.18734	0.15738	0.19838
	CrossEncoder	0.03130	0.11294	0.09774	0.14161
Month 4 (2022-09 → 2023-06)					
	BM25	0.04185	0.15812	0.13375	0.15990
	XGBoost	**0.04370**	**0.17364**	**0.14892**	**0.17953**
	NaiveBayes	0.04185	0.15813	0.13377	0.15996
	RankNet	0.02645	0.12393	0.11044	0.14332
	ListNet	0.04178	0.15800	0.13363	0.15983
	CrossEncoder	0.00484	0.01230	0.02265	0.02349
Month 5 (2022-10 → 2023-07)					
	BM25	0.05756	0.22629	0.19583	0.23561
	XGBoost	**0.05823**	**0.23284**	**0.20247**	**0.24494**
	NaiveBayes	0.05756	0.22626	0.19578	0.23559
	RankNet	0.05689	0.22478	0.19434	0.23505
	ListNet	0.05756	0.22624	0.19578	0.23556
	CrossEncoder	0.00470	0.01393	0.02687	0.02912
Month 6 (2022-11 → 2023-08)					
	BM25	0.05731	0.23297	0.20101	0.23967
	XGBoost	**0.05786**	**0.23941**	**0.20779**	**0.24863**
	NaiveBayes	0.05732	0.23292	0.20091	0.23956
	RankNet	0.05477	0.22617	0.19684	0.23649
	ListNet	0.05732	0.23287	0.20086	0.23949
	CrossEncoder	0.00976	0.04119	0.04901	0.06619

5.1 Metric Trends Across Months (6 Models, 6 Months)

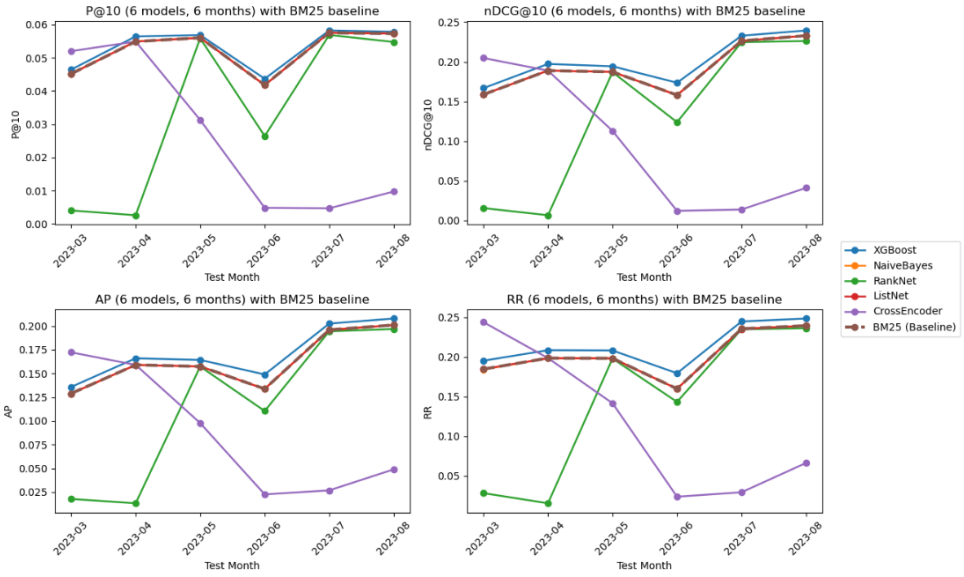

Figure 1: Trends across months for P@10, nDCG@10, AP, and RR using BM25 and five rerankers.

6 Discussion

Stable gains from XGBoost. XGBoost improves nDCG@10 in every month, indicating that learned feature-based reranking can generalize under moderate temporal drift when the candidate set is strong.

CrossEncoder instability across later months. CrossEncoder performance is excellent in Month 1 but collapses in Months 3–6. This suggests either (i) severe distribution shift and mismatched semantics across months, (ii) docno/text extraction mismatches during test indexing, or (iii) insufficiently aligned input preprocessing (length truncation, encoding, or field selection) between train and test. This behavior is treated as a key finding and also a limitation (Section 10).

RankNet/ListNet variability. RankNet improves substantially in later months relative to its very low Month 1–2 behavior, suggesting sensitivity to training signal structure and feature distribution.

7 Technical Challenges

We encountered multiple engineering challenges typical of large-scale cross-temporal IR:

- **Indexing mixed formats:** train data (TREC) vs. test data (JSON) required different ingestion paths.
- **docno mismatch and corruption:** small inconsistencies in doc identifiers can silently break neural reranking and yield artificial performance collapse.
- **GPU memory and batching:** CrossEncoder inference dominates GPU usage; batching and careful truncation are necessary.
- **Pipeline integration:** ensuring PyTerrier pipelines preserve docno fields and rank fields correctly across stages.

8 Conclusion

This project demonstrates a working two-stage BM25 + reranking pipeline for LongEval cross-temporal retrieval. Across six monthly splits, XGBoost provides consistent improvements over BM25, while transformer reranking shows strong performance in early months but instability later, emphasizing the difficulty of robust neural generalization under temporal drift and large-scale ingestion constraints.

9 Future Work

- Temporal fine-tuning or continual training for transformer rerankers.
- Robust docno/text validation tests as a standard evaluation gate.
- Dense retrieval baselines (e.g., DPR/ColBERT-style) and hybrid sparse+dense fusion.
- Ensemble reranking (tree + neural) to stabilize performance across months.

10 Limitations

- **Test qrels availability:** If evaluation on test months depends on derived or proxy judgments, reported test metrics may not reflect official leaderboard scoring.
- **Format + docno sensitivity:** CrossEncoder collapse likely indicates a failure mode tied to indexing or identifier consistency; stronger validation and sanity checks are required.
- **Compute and runtime:** Large collections constrain experimentation; full ablations (hyperparameters, feature sets, candidate depth K) were limited by time and compute.

11 Reproducibility and Research Artifacts

The full implementation of the retrieval and reranking pipeline developed in this study is publicly available through the project repository:

`https://github.com/Obiuwevwi-Lawrence/Information-Retrieval_LongEval-2025`

The repository provides a complete and self-contained record of the experimental workflow used in this paper, including data ingestion and indexing procedures, retrieval and reranking pipelines, model training code, evaluation scripts, and visualization utilities. These artifacts enable independent verification and reproduction of the reported results.

References

[1] B. Mitra and N. Craswell, "An introduction to neural information retrieval," *Foundations and Trends in Information Retrieval*, vol. 13, no. 1, pp. 1–126, 2018.

[2] A. Clément *et al.*, "Longeval: Longitudinal evaluation of model performance," in *Proceedings of the CLEF Conference*, Springer, 2023.

[3] Q. Ai, Y. Zhang, and W. B. Croft, "Understanding temporal effects in click models," *ACM Transactions on Information Systems*, vol. 39, no. 2, pp. 1–34, 2021.

[4] J. Lin, R. Nogueira, and A. Yates, "Pretrained transformers for text ranking: BERT and beyond," *Synthesis Lectures on Human Language Technologies*, vol. 14, no. 4, pp. 1–325, 2021.

[5] N. Craswell, B. Mitra, R. Nogueira, D. Cohen, and R. McDonald, "Overview of the trec 2019 deep learning track," in *Proceedings of the TREC Conference*, 2020.

[6] R. Nogueira and K. Cho, "Passage re-ranking with BERT," in *Proceedings of the 27th International Conference on Learning Representations (ICLR)*, 2019.

[7] L. Gao, Z. Dai, and J. Callan, "COIL: Revisit exact lexical match with contextualized inverted list," in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 1960–1964, 2021.

[8] J. Lu, K. Hall, X. Ma, and J. Ni, "HYRR: Hybrid infused reranking for passage retrieval," in *Proceedings of the 31st ACM International Conference on Information and Knowledge Management*, pp. 1348–1357, 2022.